Quick Commerce SQL Query Implementation

Joins, aggregate functions and subqueries

Phase II (Experiment 8)

Student name	**Aryan Kadu | Trisha Khandelwal | Siddhesh Khanolkar**
Batch	B2
UID	2025300103 | 2025300112 | 2025300114
Submission date	14/10/26
DBMS	MySQL 8.0+
GitHub Link	https://github.com/gitdorah/quick-commerce-database

Introduction

This assignment continues the Quick Commerce Inventory Database Design work by applying SQL to its MySQL schema and sample data. The queries answer practical questions about customer orders, product demand, store inventory, and stock availability. Together, they demonstrate how joins combine related records, aggregate functions summarize data, and subqueries compare rows with calculated benchmarks.

Objectives

- Retrieve order, customer, product, and dark-store details from related tables.
- Summarize orders, quantities, stock states, and estimated inventory value.
- Use scalar, nested, and correlated subqueries to identify above-average values and stock exceptions.
- Check whether the assigned store currently has enough available stock for each order line.

Database and assumptions

The implementation uses QuickCommerceDB and the Customer, DarkStore, Product, InventoryState, Order, OrderItem, and Inventory tables from the database design assignment. In MySQL, Order is enclosed in backticks because it is a reserved word. The attached implementation script contains an expanded sample dataset; all queries below target that schema.

The order-line value uses the current Product. Price because the teaching schema does not store a historical unit price in OrderItem. Inventory value is likewise an estimate at current catalogue prices, not accounting cost. Queries report the records in the sample database and do not change them.

1. Queries using joins

Query 1: Customer orders with product details

SQL concepts: INNER JOIN across Order, Customer, OrderItem, and Product; calculated column; ORDER BY

Purpose: Show each order line with customer, order status, product, quantity, unit price, and line total.

```
SELECT o.OrderID, c.Name AS CustomerName, o.OrderDate, o.Status,
       p.Name AS ProductName, oi.Quantity, p.Price AS UnitPrice,
       oi.Quantity * p.Price AS ItemTotal
FROM `Order` AS o
INNER JOIN Customer AS c ON o.CustomerID = c.CustomerID
INNER JOIN OrderItem AS oi ON o.OrderID = oi.OrderID
INNER JOIN Product AS p ON oi.ProductID = p.ProductID
ORDER BY o.OrderID, oi.OrderItemID;
```

OrderID	CustomerName	OrderDate	Status	ProductName	Quantity	UnitPrice	ItemTotal
1	Aarav Mehta	2026-09-30 09:15:00	Delivered	Amul Taaza Milk 1L	2	62.00	124.00
1	Aarav Mehta	2026-09-30 09:15:00	Delivered	Britannia Bread 400g	1	45.00	45.00
2	Riya Sharma	2026-09-30 09:22:00	Preparing	Lay's Classic Salted 50g	3	20.00	60.00
2	Riya Sharma	2026-09-30 09:22:00	Preparing	Coca Cola 750ml	2	40.00	80.00
3	Kabir Shah	2026-09-30 09:31:00	Confirmed	Amul Taaza Milk 1L	1	62.00	62.00
3	Kabir Shah	2026-09-30 09:31:00	Confirmed	Maggi 2-Minute Noodles 70g	4	15.00	60.00
4	Ananya Patel	2026-09-30 09:42:00	Out for Delivery	Tata Salt 1kg	2	30.00	60.00
4	Ananya Patel	2026-09-30 09:42:00	Out for Delivery	Surf Excel Matic 2kg	1	320.00	320.00
5	Vihaan Joshi	2026-09-30 09:55:00	Confirmed	Parle-G Biscuits 800g	2	85.00	170.00
5	Vihaan Joshi	2026-09-30 09:55:00	Confirmed	Lay's Classic Salted 50g	1	20.00	20.00

10 rows in set (0.001 sec)

Query 2: Available inventory by dark store

SQL concepts: Multiple INNER JOINs; WHERE; ORDER BY

Purpose: List sellable quantities at each store while excluding reserved, damaged, and in-transit quantities.

```
SELECT ds.DarkStoreID, ds.Location AS DarkStoreLocation,
       p.ProductID, p.Name AS ProductName,
       i.Quantity AS AvailableQuantity
FROM Inventory AS i
INNER JOIN DarkStore AS ds ON i.DarkStoreID = ds.DarkStoreID
INNER JOIN Product AS p ON i.ProductID = p.ProductID
INNER JOIN InventoryState AS s ON i.StateID = s.StateID
WHERE s.StateName = 'Available'
ORDER BY ds.Location, p.Name;
```

DarkStoreID	DarkStoreLocation	ProductID	ProductName	AvailableQuantity
1	Andheri	1	Amul Taaza Milk 1L	25
1	Andheri	2	Britannia Bread 400g	40
1	Andheri	4	Coca Cola 750ml	35
1	Andheri	3	Lay's Classic Salted 50g	60
1	Andheri	6	Maggi 2-Minute Noodles 70g	50
2	Bandra	1	Amul Taaza Milk 1L	20
2	Bandra	2	Britannia Bread 400g	30
2	Bandra	4	Coca Cola 750ml	25
2	Bandra	3	Lay's Classic Salted 50g	45
2	Bandra	5	Tata Salt 1kg	35
4	Dadar	3	Lay's Classic Salted 50g	50
4	Dadar	6	Maggi 2-Minute Noodles 70g	45
4	Dadar	8	Parle-G Biscuits 800g	25
4	Dadar	5	Tata Salt 1kg	30
3	Powai	1	Amul Taaza Milk 1L	18
3	Powai	2	Britannia Bread 400g	25
3	Powai	7	Surf Excel Matic 2kg	15
3	Powai	5	Tata Salt 1kg	40

18 rows in set (0.000 sec)

Query 3: Orders with customer and fulfillment store

SQL concepts: INNER JOIN; ORDER BY

Purpose: Identify who placed each order and which dark store is assigned to fulfill it.

```
SELECT o.OrderID, c.Name AS CustomerName,
       ds.Location AS DarkStoreLocation, o.OrderDate, o.Status
FROM `Order` AS o
INNER JOIN Customer AS c ON o.CustomerID = c.CustomerID
INNER JOIN DarkStore AS ds ON o.DarkStoreID = ds.DarkStoreID
ORDER BY o.OrderDate, o.OrderID;
```

```
+---------+--------------+-------------------+---------------------+------------------+
| OrderID | CustomerName | DarkStoreLocation | OrderDate           | Status           |
+---------+--------------+-------------------+---------------------+------------------+
|       1 | Aarav Mehta  | Andheri           | 2026-09-30 09:15:00 | Delivered        |
|       2 | Riya Sharma  | Bandra            | 2026-09-30 09:22:00 | Preparing        |
|       3 | Kabir Shah   | Andheri           | 2026-09-30 09:31:00 | Confirmed        |
|       4 | Ananya Patel | Powai             | 2026-09-30 09:42:00 | Out for Delivery |
|       5 | Vihaan Joshi | Dadar             | 2026-09-30 09:55:00 | Confirmed        |
+---------+--------------+-------------------+---------------------+------------------+
5 rows in set (0.000 sec)
```

2. Queries using aggregate functions

Query 4: Order count by customer

SQL concepts: COUNT(); LEFT JOIN; GROUP BY; ORDER BY

Purpose: Count orders for every customer, retaining customers with zero orders.

```
SELECT c.CustomerID, c.Name AS CustomerName,
       COUNT(o.OrderID) AS TotalOrders
FROM Customer AS c
LEFT JOIN `Order` AS o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.Name
ORDER BY TotalOrders DESC, c.Name;
```

```
+------------+--------------+-------------+
| CustomerID | CustomerName | TotalOrders |
+------------+--------------+-------------+
|          1 | Aarav Mehta  |           1 |
|          4 | Ananya Patel |           1 |
|          3 | Kabir Shah   |           1 |
|          2 | Riya Sharma  |           1 |
|          5 | Vihaan Joshi |           1 |
+------------+--------------+-------------+
5 rows in set (0.002 sec)
```

Query 5: Total quantity ordered by product

SQL concepts: SUM(); INNER JOIN; GROUP BY

Purpose: Add the quantities recorded on order lines for each product.

```
SELECT p.ProductID, p.Name AS ProductName,
       SUM(oi.Quantity) AS TotalQuantityOrdered
FROM Product AS p
INNER JOIN OrderItem AS oi ON p.ProductID = oi.ProductID
GROUP BY p.ProductID, p.Name
ORDER BY TotalQuantityOrdered DESC, p.Name;
```

```
+-----------+---------------------------+----------------------+
| ProductID | ProductName               | TotalQuantityOrdered |
+-----------+---------------------------+----------------------+
|         3 | Lay's Classic Salted 50g  |                    4 |
|         6 | Maggi 2-Minute Noodles 70g |                    4 |
|         1 | Amul Taaza Milk 1L        |                    3 |
|         4 | Coca Cola 750ml           |                    2 |
|         8 | Parle-G Biscuits 800g     |                    2 |
|         5 | Tata Salt 1kg             |                    2 |
|         2 | Britannia Bread 400g      |                    1 |
|         7 | Surf Excel Matic 2kg      |                    1 |
+-----------+---------------------------+----------------------+
8 rows in set (0.001 sec)
```

Query 6: Inventory quantity by stock state

SQL concepts: SUM(); INNER JOIN; GROUP BY

Purpose: Summarize units across all stores for Available, Reserved, Damaged, and In-Transit states.

```
SELECT s.StateName AS InventoryState,
       SUM(i.Quantity) AS TotalQuantity
FROM Inventory AS i
INNER JOIN InventoryState AS s ON i.StateID = s.StateID
GROUP BY s.StateID, s.StateName
ORDER BY TotalQuantity DESC;
```

```
+----------------+---------------+
| InventoryState | TotalQuantity |
+----------------+---------------+
| Available      |           613 |
| In-Transit     |            20 |
| Reserved       |            10 |
| Damaged        |             3 |
+----------------+---------------+
4 rows in set (0.000 sec)
```

Query 7: Estimated available inventory value by store

SQL concepts: SUM(); joins; WHERE; GROUP BY

Purpose: Estimate the catalogue-price value of sellable units at each dark store.

```
SELECT ds.DarkStoreID, ds.Location AS DarkStoreLocation,
       SUM(i.Quantity * p.Price) AS AvailableInventoryValue
FROM Inventory AS i
INNER JOIN DarkStore AS ds ON i.DarkStoreID = ds.DarkStoreID
INNER JOIN Product AS p ON i.ProductID = p.ProductID
INNER JOIN InventoryState AS s ON i.StateID = s.StateID
WHERE s.StateName = 'Available'
GROUP BY ds.DarkStoreID, ds.Location
ORDER BY AvailableInventoryValue DESC;
```

```
+-------------+-------------------+-------------------------+
| DarkStoreID | DarkStoreLocation | AvailableInventoryValue |
+-------------+-------------------+-------------------------+
|           3 | Powai             |                 8241.00 |
|           1 | Andheri           |                 6700.00 |
|           2 | Bandra            |                 5540.00 |
|           4 | Dadar             |                 4700.00 |
+-------------+-------------------+-------------------------+
4 rows in set (0.000 sec)
```

3. Queries using subqueries

Query 8: Products priced above average

SQL concepts: Scalar subquery; AVG(); WHERE

Purpose: Compare each catalogue price with the average price across all products.

```
SELECT p.ProductID, p.Name AS ProductName, p.Price
FROM Product AS p
WHERE p.Price > (SELECT AVG(Price) FROM Product)
ORDER BY p.Price DESC;
```

```
+-----------+----------------------+--------+
| ProductID | ProductName          | Price  |
+-----------+----------------------+--------+
|         7 | Surf Excel Matic 2kg | 320.00 |
|         8 | Parle-G Biscuits 800g |  85.00 |
+-----------+----------------------+--------+
2 rows in set (0.001 sec)
```

Query 9: Customers above the average order count

SQL concepts: Nested subquery; COUNT(); AVG(); GROUP BY; HAVING

Purpose: Find customers whose order counts exceed the average count among customers who have at least one order.

```
SELECT c.CustomerID, c.Name AS CustomerName,
       COUNT(o.OrderID) AS TotalOrders
FROM Customer AS c
INNER JOIN `Order` AS o ON c.CustomerID = o.CustomerID
GROUP BY c.CustomerID, c.Name
HAVING COUNT(o.OrderID) > (
    SELECT AVG(CustomerOrderCounts.OrderCount)
    FROM (
        SELECT COUNT(*) AS OrderCount
        FROM `Order`
        GROUP BY CustomerID
    ) AS CustomerOrderCounts
)
ORDER BY TotalOrders DESC, c.Name;
```

```
    ->      c.Name;
Empty set (0.002 sec)

mysql>
```

Query 10: Products without available stock

SQL concepts: Correlated subquery; NOT EXISTS; joins

Purpose: List products with no positive Available quantity at any dark store.

```
SELECT p.ProductID, p.Name AS ProductName
FROM Product AS p
WHERE NOT EXISTS (
    SELECT 1
    FROM Inventory AS i
    INNER JOIN InventoryState AS s ON i.StateID = s.StateID
    WHERE i.ProductID = p.ProductID
      AND s.StateName = 'Available'
      AND i.Quantity > 0
)
ORDER BY p.Name;
```

```
Empty set (0.002 sec)

mysql>
```

Query 11: Store-product stock below the average

SQL concepts: Scalar subquery; AVG(); joins; WHERE

Purpose: Flag Available inventory records whose quantities are below the average Available-record quantity across all stores.

```
SELECT ds.Location AS DarkStoreLocation,
       p.Name AS ProductName,
       i.Quantity AS AvailableQuantity
FROM Inventory AS i
INNER JOIN DarkStore AS ds ON i.DarkStoreID = ds.DarkStoreID
INNER JOIN Product AS p ON i.ProductID = p.ProductID
INNER JOIN InventoryState AS s ON i.StateID = s.StateID
WHERE s.StateName = 'Available'
  AND i.Quantity < (
      SELECT AVG(i2.Quantity)
      FROM Inventory AS i2
      INNER JOIN InventoryState AS s2 ON i2.StateID = s2.StateID
      WHERE s2.StateName = 'Available'
  )
ORDER BY i.Quantity ASC, ds.Location, p.Name;
```

```
+-------------------+-----------------------+--------------------+
| DarkStoreLocation | ProductName           | AvailableQuantity  |
+-------------------+-----------------------+--------------------+
| Powai             | Surf Excel Matic 2kg  |                 15 |
| Powai             | Amul Taaza Milk 1L    |                 18 |
| Bandra            | Amul Taaza Milk 1L    |                 20 |
| Andheri           | Amul Taaza Milk 1L    |                 25 |
| Bandra            | Coca Cola 750ml       |                 25 |
| Dadar             | Parle-G Biscuits 800g |                 25 |
| Powai             | Britannia Bread 400g  |                 25 |
| Bandra            | Britannia Bread 400g  |                 30 |
| Dadar             | Tata Salt 1kg         |                 30 |
+-------------------+-----------------------+--------------------+
9 rows in set (0.001 sec)
```

4. Inventory availability check

Query 12: Check order-line stock at the assigned store

SQL concepts: INNER JOIN; LEFT JOIN; CASE; COALESCE()

Purpose: Compare each order line's requested quantity with the Available balance for its product at the order's assigned store.

```
SELECT o.OrderID, ds.Location AS DarkStoreLocation,
       p.Name AS ProductName, oi.Quantity AS OrderedQuantity,
       COALESCE(i.Quantity, 0) AS AvailableQuantity,
       CASE
           WHEN COALESCE(i.Quantity, 0) >= oi.Quantity
               THEN 'Sufficient Stock'
           ELSE 'Insufficient Stock'
       END AS StockStatus
FROM `Order` AS o
INNER JOIN DarkStore AS ds ON o.DarkStoreID = ds.DarkStoreID
INNER JOIN OrderItem AS oi ON o.OrderID = oi.OrderID
INNER JOIN Product AS p ON oi.ProductID = p.ProductID
LEFT JOIN InventoryState AS s ON s.StateName = 'Available'
LEFT JOIN Inventory AS i
       ON i.DarkStoreID = o.DarkStoreID
      AND i.ProductID = oi.ProductID
      AND i.StateID = s.StateID
ORDER BY o.OrderID, p.Name;
```

```
+---------+-------------------+---------------------------+-----------------+-------------------+-------------------+
| OrderID | DarkStoreLocation | ProductName               | OrderedQuantity | AvailableQuantity | StockStatus       |
+---------+-------------------+---------------------------+-----------------+-------------------+-------------------+
|       1 | Andheri           | Amul Taaza Milk 1L        |               2 |                25 | Sufficient Stock |
|       1 | Andheri           | Britannia Bread 400g      |               1 |                40 | Sufficient Stock |
|       2 | Bandra            | Coca Cola 750ml           |               2 |                25 | Sufficient Stock |
|       2 | Bandra            | Lay's Classic Salted 50g  |               3 |                45 | Sufficient Stock |
|       3 | Andheri           | Amul Taaza Milk 1L        |               1 |                25 | Sufficient Stock |
|       3 | Andheri           | Maggi 2-Minute Noodles 70g|               4 |                50 | Sufficient Stock |
|       4 | Powai             | Surf Excel Matic 2kg      |               1 |                15 | Sufficient Stock |
|       4 | Powai             | Tata Salt 1kg             |               2 |                40 | Sufficient Stock |
|       5 | Dadar             | Lay's Classic Salted 50g  |               1 |                50 | Sufficient Stock |
|       5 | Dadar             | Parle-G Biscuits 800g     |               2 |                25 | Sufficient Stock |
+---------+-------------------+---------------------------+-----------------+-------------------+-------------------+
10 rows in set (0.001 sec)
```

5. Query coverage and key observations

SQL topic	Queries	Purpose
Joins	1–3, 7, 10–12	Combine records from related tables.
Aggregate functions	4–7, 9, 11	Count, sum, and average grouped or filtered data.
Subqueries	8–11	Compare records with computed values or test for matching stock.
Conditional reporting	12	Classify each order line as sufficient or insufficient.

The queries support operational review by showing order composition, demand by product, inventory distribution, and potential low-stock or unavailable products. Query 12 is a read-only diagnostic; it does not reserve stock or guarantee that a later checkout can still obtain it.

Conclusion

This implementation demonstrates how SQL joins, grouping and aggregate functions, and subqueries can answer common quick-commerce reporting questions. It connects the normalized database design to useful order and inventory insights. For checkout safety, the availability check must be followed by a conditional inventory update inside a transaction that moves units from Available to Reserved atomically; the SELECT query alone cannot prevent overselling under concurrent requests.